SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual (githublink: https://github.com/192571027/CSA0607-)
Student Reg. No.	192571027
Student Name	RITHIKA.G

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing

the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I RITHIKA.G(192571027)_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _  ___ Date: _11/09/2026_____

Problem Statement and Scope

ShopSphere is a growing e-commerce platform whose product catalogue holds n = 100,000 products today and is expected to reach n = 1,000,000 within a year. As the platform's algorithm engineer, I audit five back-end modules (M1–M5, with M3 having two versions S1 and S2), predict how each behaves as n grows, and recommend which must be redesigned. The servers execute 10^8 basic operations per second, and the nightly processing window is 10 minutes (6×10^{10} operations).

Every result below is derived by hand (basic operation → exact count → asymptotic bound → interpretation) and then verified by an instrumented Python program that executes the algorithms and counts the basic operations. Screenshots of the source code and the program output are included in the 'Screenshots' sections of this report.

Summary of results

Module	Exact count / recurrence solution	Time	Space
M1 Duplicate detector	$n(n-1)/2$ comparisons	$\Theta(n^2)$	$\Theta(1)$
M2 Rate limiter	$2^{\lceil \log_2 n \rceil} - 1$ audits ($n = 1024 \rightarrow 1023$)	$\Theta(n)$ (claim of $\Theta(n \log n)$ is wrong)	$\Theta(1)$
M3-S1 Sequential search	best 1, worst n, avg $p(n+1)/2 + (1-p)n$	$\Theta(n)$	$\Theta(1)$
M3-S2 Binary search	$T(n) = 1 + \log_2 n$ ($\lfloor \log_2 n \rfloor + 1$ in general)	$\Theta(\log n)$	Iterative $\Theta(1)$; recursive $\Theta(\log n)$
M4 current	$T(n) = 4T(n/2) + n \rightarrow 2n^2 - n$	$\Theta(n^2)$	—
M4 redesign	$T(n) = 2T(n/2) + n \rightarrow n(\log_2 n + 1)$	$\Theta(n \log n)$	—
M5 Sales analytics	$T(n) = 2T(n/2) + n \log n \rightarrow n + n \cdot L(L+1)/2$	$\Theta(n \log^2 n)$	—
Report generator	$f(n) = 3n^2 + 10n \log_2 n + 50$	$\Theta(n^2)$, not $O(n)$	—

(a) Understanding the Problem – Input Size and Basic Operation

The input size is the quantity whose growth drives the running time; the basic operation is the statement executed most often, whose count determines the order of growth.

Module	Input size	Basic operation and where it occurs	Expected growth of its count

M1 Duplicate-Listing Detector	n = number of products	Comparison of Product[i] with Product[j] (title test, then seller test) in the IF statement of the inner loop. Executed once per pair (i, j) with i < j. The loop bodies contain nothing heavier; MarkDuplicate runs only on a match.	Quadratic – every product meets every later one, so doubling n quadruples the work.
M2 API Rate Limiter	n = upper limit on the request window (number of requests)	Audit(Request[j]) inside the inner FOR loop. The inner loop runs i times, and i doubles each outer iteration (1, 2, 4, …).	Linear – a geometric series dominated by its last term; not n log n, because the inner loop does not run n times in every outer iteration.
M3-S1 Sequential Search	n = number of products in the unsorted catalogue	Comparison of the search key with the current product while scanning.	Linear – up to n comparisons, about n/2 on average for a successful search.
M3-S2 Binary (index) Search	n = number of entries in the sorted index	Comparison of the key with the middle entry; each comparison discards half of the remaining index (T(n) = T(n/2) + 1).	Logarithmic – doubling n adds only one comparison.
M4 Recommendation Engine	n = size of the customer–product data	The linear-time merge of partial results at every node of the recursion (f(n) = n unit steps), plus the recursive calls (4 or 2 sub-problems of size n/2).	Current: quadratic (four half-size subproblems). Redesign: n log n (two half-size subproblems).
M5 Sales Analytics Engine	n = number of sales records	Record comparison made while sorting by revenue at each recursion level (f(n) = n log n at the top level).	Between n log n and n^2; solved below as $n \log^2 n$.
Nightly report generator	n = number of records processed	The dominant term of $f(n) = 3n^2 + 10n \log_2 n + 50$ is the $3n^2$ pairwise-style step; the n log n and constant terms are lower-order.	Quadratic.

(b) Analysis of the Iterative Modules

M1 – Duplicate-Listing Detector

Basic operation: one comparison of Product[i] with Product[j] (the IF test). For a fixed i the inner loop j = i+1 … n−1 runs n − 1 − i times, and i runs from 0 to n − 2.

$$
\begin{aligned}
C(n) &= \Sigma_{(i = 0 \text{ to } n-2)} \, \Sigma_{(j = i+1 \text{ to } n-1)} \, 1 \\
&= \Sigma_{(i = 0 \text{ to } n-2)} \, (n - 1 - i) \\
&= (n-1) + (n-2) + \dots + 2 + 1 \qquad (k = n - 1 - i) \\
&= \Sigma_{(k = 1 \text{ to } n-1)} \, k \\
&= (n - 1) \cdot n / 2 \\
&= (n^2 - n) / 2
\end{aligned}
$$

$C(n) = n(n - 1)/2$ comparisons; Time complexity $\Theta(n^2)$; Space complexity $\Theta(1)$ auxiliary.

- Asymptotic form: $n^2/2 - n/2 \le n^2$ for $n \ge 1$ and $\ge n^2/4$ for $n \ge 2$, so $C(n) = \Theta(n^2)$. The count depends only on n (no early exit), hence best = average = worst case. (If both the title and

seller fields are tested, at most 2 field comparisons occur per pair, which only changes the constant.)

- Space: only the loop counters i, j are needed – Θ(1) extra space (the catalogue itself, Θ(n), is the input and is not counted).
- Values: n = 100,000 → 4,999,950,000 comparisons (50.0 s); n = 1,000,000 → 499,999,500,000 comparisons (83.3 min). A 10× larger catalogue costs 100× more.

M2 – API Rate Limiter

Basic operation: Audit(Request[j]). The outer loop uses i = 1, 2, 4, …, 2^t, … and continues while i < n. Let the loop run k times, i.e. i takes the values 2^0, 2^1, …, 2^(k−1). The loop stops at the first power of two that is ≥ n, so k = ceil($\log_2$ n). In the iteration where i = 2^t the inner loop runs i = 2^t times.

$$
\begin{aligned}
A(n) &= \Sigma\ (t = 0 \text{ to } k-1)\ 2^t \\
&= 2^0 + 2^1 + \ldots + 2^{(k-1)} \\
&= (2^k - 1) / (2 - 1) \qquad \text{(geometric series)} \\
&= 2^{\text{ceil}(\log_2 n)} - 1
\end{aligned}
$$

Bounding the closed form: since k = ceil($\log_2$ n), we have 2^(k−1) < n ≤ 2^k. The right inequality gives A(n) = 2^k − 1 ≥ n − 1. The left inequality (2^(k−1) ≤ n − 1 for integers) gives 2^k ≤ 2n − 2, so A(n) ≤ 2n − 3.

$$n - 1 \ \le\ A(n)\ \le\ 2n - 3 \quad \text{for all } n \ge 2 \qquad \Rightarrow \qquad A(n) = \Theta(n)$$

A(n) = 2^ceil($\log_2$ n) − 1 = Θ(n). Space complexity Θ(1).

Is the team member's Θ(n log n) claim correct?

No. The reasoning 'the outer loop runs about log n times and the inner loop runs up to n times' multiplies the worst case of each loop, but the inner loop only reaches n in the last iteration; in iteration t it runs 2^t times. The total is a geometric series dominated by its largest term, giving about n to 2n operations, not n log n. The claim n log n is only a loose upper bound (A(n) = O(n log n) is true but not tight); the statement Θ(n log n) is false because A(n) ≤ 2n is far below c·n log n for large n. The tight bound is Θ(n).

Verification for n = 1,024

Outer iterations: i = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 → k = 10
(i = 1024 is not < 1024, so the loop stops)
Audits = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1,023 = 2^{10} − 1 OK
Claim n $\log_2$ n = 1024 × 10 = 10,240 → about 10× too large

Verified by execution: the program counts exactly 1,023 Audit operations for n = 1024. For n = 1,000,000 it counts 1,048,575 (≈ 1.05 n).

(c) Solving a Recurrence by Substitution – Search Version S2

Recurrence: $T(n) = T(n/2) + 1$, $T(1) = 1$ (one comparison, then half of the index remains). Assume $n = 2^k$.

```
T(n) = T(n/2) + 1
     = [T(n/4) + 1] + 1        = T(n/4) + 2
     = [T(n/8) + 1] + 2        = T(n/8) + 3
     = [T(n/16) + 1] + 3       = T(n/16) + 4
     ⋮
     = T(n/2^i) + i            (general pattern after i substitutions)
```

The recursion bottoms out when $n/2^i = 1$, i.e. $i = \log_2 n$. Substituting:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

Proof by induction: $T(1) = 1 + \log_2 1 = 1$ OK. If $T(n/2) = 1 + \log_2(n/2) = \log_2 n$, then $T(n) = T(n/2) + 1 = \log_2 n + 1$ OK. For arbitrary n the exact worst case is $\text{floor}(\log_2 n) + 1$ (for n = 1,000,000 this is 20 comparisons).

Exact closed form: $T(n) = 1 + \log_2 n$ ($\text{floor}(\log_2 n) + 1$ for general n) $\Rightarrow$ $\Theta(\log n)$.

Implementation	Time complexity	Space complexity	Reason
Iterative binary search	Θ(log n) (best Θ(1))	Θ(1)	Only lo, hi and mid are stored; the index is searched in place.
Recursive binary search	Θ(log n) (best Θ(1))	Θ(log n)	Each call adds a stack frame; the recursion depth is $\text{floor}(\log_2 n) + 1$ (11 frames for n = 1024, 20 for $n = 10^6$).

Interpretation: the product index can grow ten-fold ($10^5 \rightarrow 10^6$) at a cost of only about 3.3 extra comparisons per search (17 → 20), which is why S2 is the right design for the live search path. The program confirms $T(n) = 1 + \log_2 n$ for $n = 2^0 \ldots 2^{20}$ and the recursion depth/worst case of 10 for n = 1,023.

(d) Applying the Master Theorem

Master Theorem for $T(n) = a \cdot T(n/b) + f(n)$, $a \geq 1$, $b > 1$, with critical exponent $n^{(\log_b a)}$:

- Case 1: $f(n) = O(n^{(\log_b a - \varepsilon)})$ for some $\varepsilon > 0$ $\Rightarrow$ $T(n) = \Theta(n^{(\log_b a)})$.
- Case 2: $f(n) = \Theta(n^{(\log_b a)})$ $\Rightarrow$ $T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$.
- Case 3: $f(n) = \Omega(n^{(\log_b a + \varepsilon)})$ and $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$ $\Rightarrow$ $T(n) = \Theta(f(n))$.

M4 – Recommendation Engine

	a	b	f(n)	n^(log_b a)	Comparison	Case	Result
Current: 4T(n/2) + n	4	2	n	n^($\log_2 4$) = n^2	f = n = O(n^(2−ε)), ε = 1	1	Θ(n^2)
Redesign: 2T(n/2) + n	2	2	n	n^($\log_2 2$) = n	f = n = Θ(n)	2	Θ(n log n)

Both designs have been solved exactly as well (T(1) = 1, n = 2^k):

Current : T(n) = 4T(n/2) + n = $2n^2 - n$ (check: T(2) = 4 + 2 = 6 = 2·4 − 2 OK)

Redesign: T(n) = 2T(n/2) + n = n($\log_2 n$ + 1) (check: T(2) = 2 + 2 = 4 = 2·(1+1) OK)

What happens when the data doubles?

Current : T(2n)/T(n) = $(8n^2 - 2n)/(2n^2 - n)$ = 4 + 2/(2n − 1) → 4

Redesign: T(2n)/T(n) = 2n($\log_2 n$ + 2) / n($\log_2 n$ + 1) = 2(L + 2)/(L + 1) → 2 (≈ 2.095 at n = 2^{20})

Current design: doubling the data multiplies the running time by 4 (Θ(n^2)). Redesign: doubling multiplies it by only slightly more than 2 (Θ(n log n)). At n = 10^6 the redesign needs about 2.1 × 10^7 operations versus 2 × 10^{12} for the current design – roughly 95,000 times fewer.

M5 – Sales Analytics Engine: T(n) = 2T(n/2) + n log n

a = 2, b = 2, f(n) = n log n, n^(log_b a) = n^($\log_2 2$) = n

Do the three standard cases apply?

- **Case 1:** needs f(n) = O(n^(1−ε)). But f(n) = n log n grows faster than n, so it fails.
- **Case 2:** needs f(n) = Θ(n). But n log n / n = log n → ∞, so f is not Θ(n); it fails.
- **Case 3:** needs f(n) = Ω(n^(1+ε)) for a fixed ε > 0. But f(n)/n^(1+ε) = log n / n^ε → 0 for every ε > 0, so f is larger than n only by a log factor, not polynomially; it fails (the regularity condition is not even reached).

Conclusion: none of the three standard cases applies – f(n) lies in the gap between Case 2 and Case 3. The extended Master Theorem (or a recursion tree) is needed.

Extended Master Theorem

If f(n) = Θ(n^(log_b a) · $\log^k$ n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n). Here f(n) = n log n = Θ(n · $\log^1$ n), so k = 1:

T(n) = Θ(n · log^(1+1) n) = Θ(n $\log^2$ n)

Verification by recursion tree

Let L = $\log_2$ n. Level i (i = 0 … L−1): 2^i sub-problems of size $n/2^i$

cost per sub-problem = $(n/2^i)\cdot\log_2(n/2^i)$ = $(n/2^i)(L - i)$

cost of level i $= 2^i \cdot (n/2^i)(L - i) = n(L - i)$

Internal levels: $n \cdot \Sigma\ (L - i) = n \cdot (L + (L-1) + \ldots + 1) = n \cdot L(L+1)/2$

Leaves: n sub-problems of size 1, each $T(1) = 1 \rightarrow n$

$T(n) = n + n \cdot L(L+1)/2 = n + n \cdot \log_2 n \cdot (\log_2 n + 1)/2$

$T(n) = \Theta(n \log^2 n)$; the program's direct evaluation of the recurrence equals $n + nL(L+1)/2$ exactly (e.g. $n = 16 \rightarrow 176$) and $T(n)/(n \log^2 n)$ settles near 0.5.

(e) Best, Average and Worst Case Analysis – S1 and S2

S1 – Sequential search (unsorted catalogue)

Case	Input condition	Comparisons
Best	Searched product is the first item scanned	$1 \rightarrow \Theta(1)$
Worst	Searched product is the last item, or is absent	$n \rightarrow \Theta(n)$
Average (successful)	Product present, equally likely at each of n positions	$(n + 1)/2 \rightarrow \Theta(n)$
Average (unsuccessful)	Product absent	n

Average with probability p that the product exists. If it exists, position i (1 … n) is equally likely, costing i comparisons; if absent, all n items are compared.

$$A(n, p) = p \cdot [\ (1/n)\ \Sigma\ (i=1..n)\ i\ ]\ +\ (1 - p) \cdot n$$
$$= p \cdot (n + 1)/2\ +\ (1 - p) \cdot n$$
$$= n - p \cdot n/2 + p/2\ =\ n(1 - p/2) + p/2$$

Evaluation for n = 1,000,000 and p = 0.5:

$$A = 0.5 \times (1{,}000{,}001 / 2) + 0.5 \times 1{,}000{,}000$$
$$= 0.5 \times 500{,}000.5 + 500{,}000$$
$$= 250{,}000.25 + 500{,}000 = 750{,}000.25 \text{ comparisons}$$

$A(1{,}000{,}000, 0.5) = 750{,}000.25$ comparisons (≈ 7.5 ms at 10^8 comparisons/s). Boundary checks: $p = 1 \rightarrow (n+1)/2$; $p = 0 \rightarrow n$.

S2 – Binary search (sorted index)

Case	Input condition	Comparisons
Best	Key equals the middle entry of the whole index	$1 \rightarrow \Theta(1)$

Average	Key present, all positions equally likely (most keys sit in the lowest tree levels)	$\approx \log_2 n - 1 \rightarrow \Theta(\log n)$ (18.95 measured for $n = 10^6$)
Worst	Key found only at the last level, or absent	$\text{floor}(\log_2 n) + 1 \rightarrow \Theta(\log n)$ (20 for $n = 10^6$)

Average reasoning: the comparison tree has 2^(j–1) keys at level j, so for $n = 2^k - 1$ the average successful search costs $\Sigma\, j \cdot 2^{(j-1)} / n = ((k-1)2^k + 1)/n \approx \log_2 n - 1$. The program measured 18.951 comparisons averaged over all 10^6 keys.

Interpretation: S1 needs about 750,000 comparisons per lookup at p = 0.5, S2 needs at most 20 – around 37,500 times fewer on average – at the price of keeping the index sorted.

(f) Proof of an Asymptotic Bound – Report Generator

Claim: $f(n) = 3n^2 + 10n \log_2 n + 50 = \Theta(n^2)$. By definition we must find constants $c_1, c_2 > 0$ and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

Lower bound ($f(n) = \Omega(n^2)$)

For $n \ge 1$, $\log_2 n \ge 0$, so $10n \log_2 n \ge 0$ and $50 > 0$. Therefore $f(n) \ge 3n^2$. Take $c_1 = 3$.

Upper bound ($f(n) = O(n^2)$)

For $n \ge 1$: $\log_2 n \le n$, hence $10n \log_2 n \le 10n^2$; and $50 \le 50n^2$. Therefore

$$f(n) = 3n^2 + 10n \log_2 n + 50 \;\le\; 3n^2 + 10n^2 + 50n^2 \;=\; 63n^2$$

Constants: $c_1 = 3$, $c_2 = 63$, $n_0 = 1 \;\Rightarrow\; 3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$, hence $f(n) = \Theta(n^2)$.

A tighter pair also works: $c_1 = 3$, $c_2 = 4$, $n_0 = 60$, because $10n \log_2 n + 50 \le n^2$ once $n \ge 60$ (at n = 60: 3,594 ≤ 3,600; at n = 59 it fails: 3,521 > 3,481, so $n_0 = 60$ is the smallest). The program checks both pairs numerically up to n = 20,000.

f(n) is not O(n)

Suppose $f(n) = O(n)$. Then there exist $c > 0$ and n_1 such that $f(n) \le c \cdot n$ for all $n \ge n_1$. But $f(n) \ge 3n^2$, so $3n^2 \le c \cdot n$, i.e. $n \le c/3$ for every $n \ge n_1$. This is false for any $n > \max(n_1, c/3)$ – a contradiction. Equivalently, $f(n)/n \ge 3n \to \infty$. Hence $f(n) \notin O(n)$.

Numerical check (program): $f(n)/n^2 \to 3.0002$ at $n = 10^6$ (bounded), while $f(n)/n = 3{,}000{,}199$ grows without bound. For $c = 10^6$ the first violation is n = 333,334, where f(n) = 333,395,822,203 > c·n = 333,334,000,000.

(g) Scalability Decision for n = 1,000,000

Assumptions: 10^8 basic operations per second; nightly window 10 min = 600 s = 6×10^{10} operations. Operation counts use the exact closed forms from parts (b)–(f); the logarithm is base 2 ($\log_2 10^6 \approx 19.93$).

Module	Complexity	Operations	Time	10-min window
M1 Duplicate detector	$\Theta(n^2)$	4.99999×10^{11}	83.3 min	FAIL
M2 Rate limiter	$\Theta(n)$	1,048,575	10.5 ms	PASS
S2 Binary search (per query)	$\Theta(\log n)$	20	0.2 µs	PASS
M4 current ($2n^2 - n$)	$\Theta(n^2)$	2.0×10^{12}	5.56 h	FAIL
M4 redesign ($n(\log n + 1)$)	$\Theta(n \log n)$	2.09×10^7	209 ms	PASS
M5 Sales analytics	$\Theta(n \log^2 n)$	2.10×10^8	2.10 s	PASS
Report generator f(n)	$\Theta(n^2)$	3.0×10^{12}	8.33 h	FAIL

Growth from n = 100,000 to n = 1,000,000 (10× catalogue)

Module	Operations at 10^5	Operations at 10^6	Growth factor
M1	4,999,950,000	499,999,500,000	100×
M2	131,071	1,048,575	8×
M4 current	19,999,900,000	1,999,999,000,000	100×
M4 redesign	1,760,964	20,931,569	11.9×
M5	14,724,490	209,599,497	14.2×
Report generator	30,016,609,690	3,000,199,315,736	100×

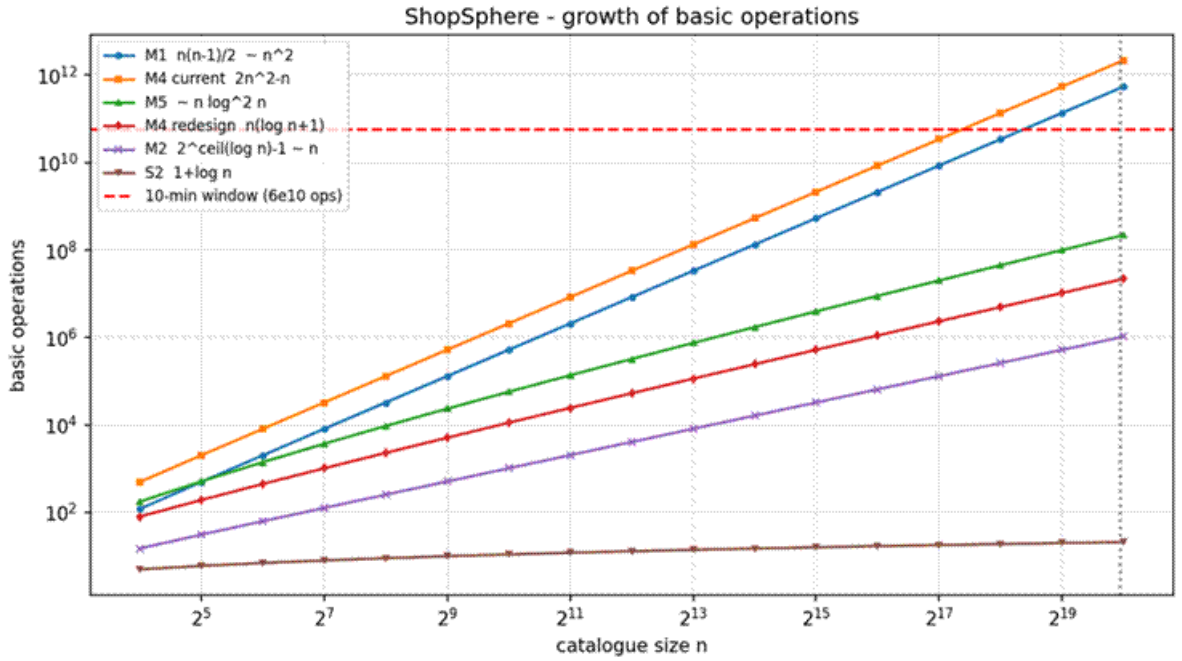


Figure 1 – Growth of basic operations against catalogue size (log–log axes); the red line is the 10-minute window.

Modules that fail the 10-minute window

- **M1 Duplicate-Listing Detector –** ≈ 5 × 10^{11} operations, 83 min: 8.3× over the 10-minute budget.
- **M4 Recommendation Engine (current design) –** ≈ 2 × 10^{12} operations, 5.6 h: 33× over the budget.
- **Nightly report generator –** ≈ 3 × 10^{12} operations, 8.3 h: also quadratic and also fails, so it should be redesigned alongside M1 and M4.

M2 (10 ms), S2 (0.2 µs per query), the M4 redesign (0.21 s) and M5 (2.1 s) all finish far inside the window; at n = 10^6 M5's Θ(n log² n) is acceptable.

Recommendations

- **Redesign M1 (Θ(n²) → Θ(n) expected or Θ(n log n)).** Build a hash set keyed on (normalised title, seller) and flag a product as a duplicate when its key is already present: one pass, about n operations (≈ 10 ms), at the cost of Θ(n) extra memory. If memory is tight, sort by the (title, seller) key (Θ(n log n) ≈ 2.1 × 10^7 operations ≈ 0.21 s) and compare only neighbours. Either option also makes the nightly job incremental (check only newly uploaded listings).
- **Redesign M4: remove the replication-driven fourth sub-problem.** The proposed T(n) = 2T(n/2) + n design is Θ(n log n) and cuts the cost from 2 × 10^{12} to 2.1 × 10^7 operations (5.56 h → 0.21 s). Fault-tolerance can be kept at the storage layer (replicated copies of the data) instead of replicating the computation.

- **Redesign the report generator (Θ(n²)).** Identify the quadratic step and replace it with sorting/hash-based aggregation (Θ(n log n) or Θ(n)); at 8.3 h it cannot fit any nightly window.
- **Keep S2 and retire S1 for live search.** S1 averages 750,000 comparisons per lookup (p = 0.5) versus at most 20 for S2; maintain the sorted (or hash) index incrementally.
- **Keep M2; optionally optimise M5.** M5 is fine at 2.1 s; sorting once with a Θ(n log n) merge sort instead of re-sorting at every level would bring it to Θ(n log n) if the catalogue grows well beyond 10^6.

Implementation and Execution

Language: Python 3 (standard library only; matplotlib is optional and used only for Figure 1). File: Source_Code/shopsphere_audit.py.

- Each module is implemented as in the question and instrumented to return its basic-operation count: m1_duplicate_detector, m2_rate_limiter, s1_sequential_search, s2_binary_search_iter and s2_binary_search_rec.
- The recurrences T_s2, T_m4_cur, T_m4_new and T_m5 are evaluated directly for $n = 2^k$ and compared with the closed forms derived above.
- Parts (b)–(g) each have a function (part_b … part_g) that prints measured-versus-derived values; part_g prints the scalability table with PASS/FAIL against the 6×10^{10} operation window.

How to run:

```
cd CO1_AT1/Question_1/Source_Code
python shopsphere_audit.py              # prints all results, saves ../Supporting_Files/growth_chart.png
```

Verification summary

- M1: measured n(n−1)/2 exactly for n = 5, 10, 100, 500, 1000; duplicate demo flags the repeated listing.
- M2: measured 2^ceil($\log_2$ n) − 1 for every tested n, including 1,023 audits at n = 1,024 and 1,048,575 at n = 10^6.
- S2: recurrence equals 1 + $\log_2$ n for n = 2^0 … 2^{20}; the worst case and recursion depth at n = 1,023 are both 10.
- M4/M5: recurrences equal $2n^2 - n$, n($\log_2$ n + 1) and n + nL(L+1)/2 exactly; doubling ratios are 4.00 and ≈ 2.1.
- S1: Monte-Carlo estimate (n = 2000, p = 0.5, 60,000 trials) 1,501.6 against formula 1,500.2.

Screenshots of the Source Code

```
"""
CSA06 - Design and Analysis of Algorithms
CO1 - Assessment Tool 1 : Scalability Audit of "ShopSphere"

Implements every module (M1-M5, S1, S2), counts basic operations by actually
executing the algorithms, checks the counts against the derived closed forms,
and produces the scalability table for n = 1,000,000.

Assumption (given in the question): the servers execute 10^8 basic ops / second.
Run :  python shopsphere_audit.py
"""
import math
import os
import random

OPS_PER_SEC = 10 ** 8
WINDOW_SEC = 10 * 60            # nightly processing window = 10 minutes
WINDOW_OPS = OPS_PER_SEC * WINDOW_SEC


# ---------------------------------------------------------------------------
# Module implementations (instrumented: each returns the basic-operation count)
# ---------------------------------------------------------------------------
def m1_duplicate_detector(products):
    """M1: compare every product with every later product.
    Basic operation = one pair comparison (title test)."""
    n, comparisons, dups = len(products), 0, []
    for i in range(0, n - 1):
        for j in range(i + 1, n):
            comparisons += 1
            if products[i][0] == products[j][0] and products[i][1] == products[j][1]:
                dups.append(j)
    return comparisons, dups


def m2_rate_limiter(n):
    """M2: audit windows of doubling size.  Basic operation = Audit(Request[j])."""
    audits, i = 0, 1
    while i < n:
        for _ in range(0, i):
            audits += 1
        i *= 2
    return audits


def s1_sequential_search(arr, key):
    """S1: sequential scan.  Returns (index or -1, comparisons)."""
    comps = 0
    for idx, v in enumerate(arr):
        comps += 1
        if v == key:
            return idx, comps
```

Screenshot 1 – Source code (shopsphere_audit.py), part 1 of 7

```
    return -1, comps


def s2_binary_search_iter(arr, key):
    """S2 (iterative): O(1) extra space."""
    lo, hi, comps = 0, len(arr) - 1, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        comps += 1
        if arr[mid] == key:
            return mid, comps
        if key < arr[mid]:
            hi = mid - 1
        else:
            lo = mid + 1
    return -1, comps


def s2_binary_search_rec(arr, key, lo=0, hi=None, depth=1):
    """S2 (recursive): O(log n) stack frames.  Returns (idx, comps, max_depth)."""
    if hi is None:
        hi = len(arr) - 1
    if lo > hi:
        return -1, 0, depth - 1
    mid = (lo + hi) // 2
    if arr[mid] == key:
        return mid, 1, depth
    if key < arr[mid]:
        r = s2_binary_search_rec(arr, key, lo, mid - 1, depth + 1)
    else:
        r = s2_binary_search_rec(arr, key, mid + 1, hi, depth + 1)
    return r[0], r[1] + 1, r[2]


# ---------------------------------------------------------------------------
# Recurrences (evaluated directly for n = power of two)
# ---------------------------------------------------------------------------
def T_s2(n):
    return 1 if n == 1 else T_s2(n // 2) + 1


def T_m4_cur(n):
    return 1 if n == 1 else 4 * T_m4_cur(n // 2) + n


def T_m4_new(n):
    return 1 if n == 1 else 2 * T_m4_new(n // 2) + n


def T_m5(n):
    return 1 if n == 1 else 2 * T_m5(n // 2) + n * math.log2(n)

```

Screenshot 2 – Source code (shopsphere_audit.py), part 2 of 7

```
105
106 # ---------------------------------------------------------------------------
107 # Closed forms derived in the report
108 # ---------------------------------------------------------------------------
109 def m1_formula(n):
110     return n * (n - 1) // 2
111
112
113 def m2_formula(n):
114     return 2 ** math.ceil(math.log2(n)) - 1
115
116
117 def s2_formula(n):
118     return int(math.log2(n)) + 1
119
120
121 def m4_cur_formula(n):
122     return 2 * n * n - n
123
124
125 def m4_new_formula(n):
126     return n * math.log2(n) + n
127
128
129 def m5_formula(n):
130     L = math.log2(n)
131     return n + n * L * (L + 1) / 2
132
133
134 def f_report(n):
135     return 3 * n * n + 10 * n * math.log2(n) + 50
136
137
138 def s1_average(n, p):
139     return p * (n + 1) / 2 + (1 - p) * n
140
141
142 def fmt_time(sec):
143     if sec < 1e-3:
144         return f"{sec * 1e6:.2f} us"
145     if sec < 1:
146         return f"{sec * 1e3:.2f} ms"
147     if sec < 120:
148         return f"{sec:.2f} s"
149     if sec < 7200:
150         return f"{sec / 60:.1f} min"
151     return f"{sec / 3600:.2f} h"
152
153
154 def banner(t):
155     print("\n" + "=" * 78 + f"\n{t}\n" + "=" * 78)
156
```

Screenshot 3 – Source code (shopsphere_audit.py), part 3 of 7

```
158 # ---------------------------------------------------------------------------
159 def part_b():
160     banner("PART (b)  Iterative modules M1 and M2 : measured vs. derived counts")
161     print("M1: comparisons = n(n-1)/2")
162     print(f"{'n':>8} {'measured':>14} {'formula':>14}  match")
163     for n in (5, 10, 100, 500, 1000):
164         prods = [(f"t{k}", f"s{k}") for k in range(n)]
165         c, _ = m1_duplicate_detector(prods)
166         print(f"{n:>8} {c:>14} {m1_formula(n):>14}  {c == m1_formula(n)}")
167     prods = [("A", "S1"), ("B", "S1"), ("A", "S1"), ("C", "S2"), ("A", "S2")]
168     _, d = m1_duplicate_detector(prods)
169     print("Duplicate demo (rows 0 and 2 are the same title+seller): flagged ->", d)
170
171     print("\nM2: Audit operations = 2^ceil(log2 n) - 1")
172     print(f"{'n':>10} {'measured':>12} {'formula':>12} {'n log2 n':>14}  match")
173     for n in (2, 3, 8, 100, 1000, 1024, 1025, 100000):
174         a = m2_rate_limiter(n)
175         print(f"{n:>10} {a:>12} {m2_formula(n):>12} {n * math.log2(n):>14.0f}  {a == m2_formula(n)}")
176     a = m2_rate_limiter(1024)
177     print(f"\nVerification n=1024: outer iterations = {int(math.log2(1024))} "
178           f"(i = 1,2,...,512), Audit ops = {a}  (= 2^10 - 1 = 1023)")
179     print("Ratio audits/n for growing n (stays between 1 and 2 -> Theta(n)):")
180     for n in (10 ** 3, 10 ** 4, 10 ** 5, 10 ** 6):
181         r = m2_rate_limiter(n)
182         print(f"   n={n:>8}: audits={r:>8}  audits/n={r / n:.3f}")
183
184
185 def part_c():
186     banner("PART (c)  Recurrence for S2 : T(n)=T(n/2)+1, T(1)=1")
187     print(f"{'n':>10} {'T(n) recurrence':>16} {'1+log2 n':>10}  match")
188     for k in (0, 1, 4, 10, 20):
189         n = 2 ** k
190         print(f"{n:>10} {T_s2(n):>16} {1 + k:>10}  {T_s2(n) == 1 + k}")
191     arr = list(range(1, 1024))                      # n = 2^10 - 1
192     worst = max(s2_binary_search_iter(arr, k)[1] for k in arr)
193     _, c, depth = s2_binary_search_rec(arr, 1)
194     print(f"\nn=1023: worst-case comparisons (iterative) = {worst}; "
195           f"recursion depth (stack frames) = {depth}")
196     print("Space: iterative O(1) (lo, hi, mid) ; recursive O(log n) call stack")
197
198
199 def part_d():
200     banner("PART (d)  Master theorem : M4 and M5")
201     print("M4 current : T(n)=4T(n/2)+n   a=4 b=2 f=n   n^log2(4)=n^2  -> Case 1 -> Theta(n^2)")
202     print("M4 redesign: T(n)=2T(n/2)+n   a=2 b=2 f=n   n^log2(2)=n    -> Case 2 -> Theta(n log n)")
203     print(f"\n{'n':>10} {'T_cur':>16} {'2n^2-n':>16} {'T_new':>14} {'n(log n+1)':>14}")
204     for k in (4, 8, 12, 16):
205         n = 2 ** k
206         print(f"{n:>10} {T_m4_cur(n):>16} {m4_cur_formula(n):>16} "
207               f"{T_m4_new(n):>14} {m4_new_formula(n):>14.0f}")
208     print("\nEffect of doubling the data (ratio T(2n)/T(n)):")
```

Screenshot 4 – Source code (shopsphere_audit.py), part 4 of 7

```
    print(f"{'n':>10} {'current':>10} {'redesign':>10}")
    for k in (10, 14, 18, 20):
        n = 2 ** k
        print(f"{n:>10} {T_m4_cur(2 * n) / T_m4_cur(n):>10.3f} {T_m4_new(2 * n) / T_m4_new(n):>10.3f}")
    print("\nM5: T(n)=2T(n/2)+n log n   a=2 b=2  n^log_b a = n   f(n)=n log n")
    print("   Case 1 fails (f is bigger than n), Case 2 (k=0) fails (extra log),")
    print("   Case 3 fails (n log n is NOT Omega(n^(1+eps))). Extended MT, k=1 -> Theta(n log^2 n)")
    print(f"\n{'n':>10} {'T_m5':>16} {'n+n*L(L+1)/2':>16} {'T/(n log^2 n)':>14}")
    for k in (4, 8, 12, 16, 20):
        n = 2 ** k
        print(f"{n:>10} {T_m5(n):>16.1f} {m5_formula(n):>16.1f} {T_m5(n) / (n * k * k):>14.3f}")


def part_e():
    banner("PART (e)  Best / average / worst cases of S1 and S2")
    n = 1_000_000
    arr = list(range(1, n + 1))
    print(f"n = {n:,}")
    print(f"S1 best  (key at index 0)     : {s1_sequential_search(arr, 1)[1]:>9,} comparisons")
    print(f"S1 worst (key last / absent)  : {s1_sequential_search(arr, n)[1]:>9,} / "
          f"{s1_sequential_search(arr, -5)[1]:,} comparisons")
    print("S1 average, successful search : (n+1)/2 = {:,.1f}".format((n + 1) / 2))
    print("S1 average, unsuccessful      : n = {:,}".format(n))
    print("\nS1 average A(n,p) = p(n+1)/2 + (1-p)n")
    for p in (0.0, 0.25, 0.5, 0.75, 1.0):
        print(f"   p={p:<4}: A = {s1_average(n, p):>14,.2f}")
    print(f"\n>>> n=1,000,000, p=0.5  ->  A = {s1_average(n, 0.5):,.2f} comparisons")

    # empirical check of the formula at a smaller n with random queries
    random.seed(7)
    m, p, trials = 2000, 0.5, 60000
    a2 = list(range(m))
    tot = 0
    for _ in range(trials):
        key = random.randrange(m) if random.random() < p else -1
        tot += s1_sequential_search(a2, key)[1]
    print(f"Monte-Carlo check (n={m}, p={p}, {trials} trials): "
          f"measured {tot / trials:,.1f}  vs  formula {s1_average(m, p):,.1f}")

    print("\nS2 (binary search) on sorted index, n = 1,000,000")
    counts = [s2_binary_search_iter(arr, k)[1] for k in range(1, n + 1)]
    print(f"S2 best  (key = middle element): {s2_binary_search_iter(arr, arr[(n - 1) // 2])[1]} comparison")
    print(f"S2 average (successful, all keys): {sum(counts) / len(counts):.3f} comparisons")
    print(f"S2 worst : {max(counts)} comparisons   (floor(log2 n)+1 = {s2_formula(n)})")
    print(f"S2 unsuccessful search          : {s2_binary_search_iter(arr, 0)[1]} comparisons")


def part_f():
    banner("PART (f)  f(n)=3n^2+10n log2 n+50  is Theta(n^2)")
    c1, c2, n0 = 3, 63, 1
    ok = all(c1 * n * n <= f_report(n) <= c2 * n * n for n in range(n0, 20001))
    print(f"constants c1={c1}, c2={c2}, n0={n0}: c1 n^2 <= f(n) <= c2 n^2 for 1<=n<=20000 : {ok}")
```

Screenshot 5 – Source code (shopsphere_audit.py), part 5 of 7

```
    c2b, n0b = 4, 60
    ok2 = all(c1 * n * n <= f_report(n) <= c2b * n * n for n in range(n0b, 20001))
    print(f"tighter pair c1=3, c2=4, n0=60                                  : {ok2}"
          f"   (n=59 fails: f={f_report(59):.1f} > 4n^2={4 * 59 * 59})")
    print(f"\n{'n':>8} {'f(n)':>18} {'f(n)/n^2':>10} {'f(n)/n':>16}")
    for n in (10, 100, 1000, 10 ** 4, 10 ** 5, 10 ** 6):
        print(f"{n:>8} {f_report(n):>18,.1f} {f_report(n) / n ** 2:>10.4f} {f_report(n) / n:>16,.1f}")
    print("f(n)/n^2 -> 3 (bounded)  => Theta(n^2);  f(n)/n -> infinity => f(n) is not O(n)")
    print("Disproof of O(n): for any c, n > c/3 gives f(n) >= 3n^2 > c n.")
    for c in (10, 1000, 10 ** 6):
        n = c // 3 + 1
        print(f"   c={c:>8}: n={n:>8} -> f(n)={f_report(n):,.0f} > c*n={c * n:,}  : {f_report(n) > c * n}")


def part_g():
    banner("PART (g)  Scalability decision  (10^8 ops/s, window = 10 min = 6x10^10 ops)")
    n = 1_000_000
    rows = [
        ("M1 Duplicate detector", m1_formula(n), "Theta(n^2)"),
        ("M2 Rate limiter", m2_formula(n), "Theta(n)"),
        ("S2 Binary search (1 query)", s2_formula(n), "Theta(log n)"),
        ("M4 Recommender - current", m4_cur_formula(n), "Theta(n^2)"),
        ("M4 Recommender - redesign", m4_new_formula(n), "Theta(n log n)"),
        ("M5 Sales analytics", m5_formula(n), "Theta(n log^2 n)"),
        ("Report generator f(n)", f_report(n), "Theta(n^2)"),
    ]
    print(f"n = {n:,}   window budget = {WINDOW_OPS:,} operations\n")
    print(f"{'Module':<30}{'Complexity':<18}{'Operations':>20}{'Time':>12}  Verdict")
    print("-" * 92)
    for name, ops, cx in rows:
        t = ops / OPS_PER_SEC
        print(f"{name:<30}{cx:<18}{ops:>20,.0f}{fmt_time(t):>12}  "
              f"{'PASS' if ops <= WINDOW_OPS else 'FAIL'}")
    print("\nGrowth when the catalogue grows 100,000 -> 1,000,000 (10x):")
    print(f"{'Module':<30}{'ops @1e5':>18}{'ops @1e6':>18}{'ratio':>9}")
    n1 = 100_000
    gr = [("M1", m1_formula), ("M2", m2_formula), ("M4 current", m4_cur_formula),
          ("M4 redesign", m4_new_formula), ("M5", m5_formula), ("Report f(n)", f_report)]
    for name, fn in gr:
        a, b = fn(n1), fn(n)
        print(f"{name:<30}{a:>18,.0f}{b:>18,.0f}{b / a:>9.1f}")
    print("\nRedesign estimate for M1 (sort by (title,seller) key, then scan neighbours):")
    ops = n * math.log2(n) + n
    print(f"   ~ n log2 n + n = {ops:,.0f} ops = {fmt_time(ops / OPS_PER_SEC)}  "
          f"(hash-set variant: ~n ops, Theta(n) expected)")


def make_chart(path):
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
```

Screenshot 6 – Source code (shopsphere_audit.py), part 6 of 7

```
313     except ImportError:
314         print("matplotlib not installed - chart skipped")
315         return
316     ns = [2 ** k for k in range(4, 21)]
317     fig, ax = plt.subplots(figsize=(9, 5.2))
318     ax.plot(ns, [m1_formula(n) for n in ns], label="M1  n(n-1)/2  ~ n^2", marker="o", ms=3)
319     ax.plot(ns, [m4_cur_formula(n) for n in ns], label="M4 current  2n^2-n", marker="s", ms=3)
320     ax.plot(ns, [m5_formula(n) for n in ns], label="M5  ~ n log^2 n", marker="^", ms=3)
321     ax.plot(ns, [m4_new_formula(n) for n in ns], label="M4 redesign  n(log n+1)", marker="d", ms=3)
322     ax.plot(ns, [m2_formula(n) for n in ns], label="M2  2^ceil(log n)-1 ~ n", marker="x", ms=4)
323     ax.plot(ns, [s2_formula(n) for n in ns], label="S2  1+log n", marker="v", ms=3)
324     ax.axhline(WINDOW_OPS, color="red", ls="--", label="10-min window (6e10 ops)")
325     ax.axvline(1_000_000, color="grey", ls=":")
326     ax.set_xscale("log", base=2)
327     ax.set_yscale("log")
328     ax.set_xlabel("catalogue size n")
329     ax.set_ylabel("basic operations")
330     ax.set_title("ShopSphere - growth of basic operations")
331     ax.legend(fontsize=8)
332     ax.grid(alpha=.3)
333     fig.tight_layout()
334     fig.savefig(path, dpi=150)
335     print(f"chart saved -> {path}")
336
337
338 if __name__ == "__main__":
339     print("ShopSphere Scalability Audit  -  CSA06 CO1 AT1")
340     part_b()
341     part_c()
342     part_d()
343     part_e()
344     part_f()
345     part_g()
346     here = os.path.dirname(os.path.abspath(__file__))
347     make_chart(os.path.join(here, "..", "Supporting_Files", "growth_chart.png"))
```

Screenshot 7 – Source code (shopsphere_audit.py), part 7 of 7

Screenshots of the Program Output

```
================================================================================
PART (b)  Iterative modules M1 and M2 : measured vs. derived counts
================================================================================
M1: comparisons = n(n-1)/2
       n       measured        formula  match
       5             10             10  True
      10             45             45  True
     100           4950           4950  True
     500         124750         124750  True
    1000         499500         499500  True
Duplicate demo (rows 0 and 2 are the same title+seller): flagged -> [2]

M2: Audit operations = 2^ceil(log2 n) - 1
         n     measured      formula       n log2 n  match
         2            1            1              2  True
         3            3            3              5  True
         8            7            7             24  True
       100          127          127            664  True
      1000         1023         1023           9966  True
      1024         1023         1023          10240  True
      1025         2047         2047          10251  True
    100000       131071       131071        1660964  True

Verification n=1024: outer iterations = 10 (i = 1,2,...,512), Audit ops = 1023  (= 2^10 - 1 = 1023)
Ratio audits/n for growing n (stays between 1 and 2 -> Theta(n)):
   n=    1000: audits=    1023  audits/n=1.023
   n=   10000: audits=   16383  audits/n=1.638
   n=  100000: audits=  131071  audits/n=1.311
   n= 1000000: audits= 1048575  audits/n=1.049
```

Output – part (b): M1 and M2 measured vs. derived counts, n = 1,024 verification

```
================================================================================
PART (c)  Recurrence for S2 : T(n)=T(n/2)+1, T(1)=1
================================================================================
         n  T(n) recurrence   1+log2 n  match
         1                1          1  True
         2                2          2  True
        16                5          5  True
      1024               11         11  True
   1048576               21         21  True

n=1023: worst-case comparisons (iterative) = 10; recursion depth (stack frames) = 10
Space: iterative O(1) (lo, hi, mid) ; recursive O(log n) call stack
```

Output – part (c): S2 recurrence verification and space usage

```
================================================================================
PART (d)  Master theorem : M4 and M5
================================================================================
M4 current : T(n)=4T(n/2)+n   a=4 b=2 f=n   n^log2(4)=n^2  -> Case 1 -> Theta(n^2)
M4 redesign: T(n)=2T(n/2)+n   a=2 b=2 f=n   n^log2(2)=n    -> Case 2 -> Theta(n log n)

         n            T_cur          2n^2-n          T_new     n(log n+1)
        16              496             496             80             80
       256           130816          130816           2304           2304
      4096         33550336        33550336          53248          53248
     65536       8589869056      8589869056        1114112        1114112

Effect of doubling the data (ratio T(2n)/T(n)):
         n    current   redesign
      1024      4.001      2.182
     16384      4.000      2.133
    262144      4.000      2.105
   1048576      4.000      2.095

M5: T(n)=2T(n/2)+n log n   a=2 b=2  n^log_b a = n   f(n)=n log n
   Case 1 fails (f is bigger than n), Case 2 (k=0) fails (extra log),
   Case 3 fails (n log n is NOT Omega(n^(1+eps))). Extended MT, k=1 -> Theta(n log^2 n)

         n            T_m5     n+n*L(L+1)/2  T/(n log^2 n)
        16           176.0            176.0          0.688
       256          9472.0           9472.0          0.578
      4096        323584.0         323584.0          0.549
     65536       8978432.0        8978432.0          0.535
   1048576     221249536.0      221249536.0          0.527
```

Output – part (d): Master Theorem cases, exact solutions, doubling ratios

```
================================================================================
PART (e)  Best / average / worst cases of S1 and S2
================================================================================
n = 1,000,000
S1 best  (key at index 0)     :         1 comparisons
S1 worst (key last / absent)  : 1,000,000 / 1,000,000 comparisons
S1 average, successful search : (n+1)/2 = 500,000.5
S1 average, unsuccessful      : n = 1,000,000

S1 average A(n,p) = p(n+1)/2 + (1-p)n
   p=0.0 : A =   1,000,000.00
   p=0.25: A =     875,000.12
   p=0.5 : A =     750,000.25
   p=0.75: A =     625,000.38
   p=1.0 : A =     500,000.50

>>> n=1,000,000, p=0.5  ->  A = 750,000.25 comparisons
Monte-Carlo check (n=2000, p=0.5, 60000 trials): measured 1,501.6  vs  formula 1,500.2

S2 (binary search) on sorted index, n = 1,000,000
S2 best  (key = middle element): 1 comparison
S2 average (successful, all keys): 18.951 comparisons
S2 worst : 20 comparisons   (floor(log2 n)+1 = 20)
S2 unsuccessful search           : 19 comparisons
```

Output – part (e): best/average/worst cases, S1 average for n = 10^6, p = 0.5

```
======================================================================
PART (f)  f(n)=3n^2+10n log2 n+50  is Theta(n^2)
======================================================================
constants c1=3, c2=63, n0=1: c1 n^2 <= f(n) <= c2 n^2 for 1<=n<=20000 : True
tighter pair c1=3, c2=4, n0=60                                        : True   (n=59 fails: f=13963.8 > 4n^2=13924)

       n              f(n)   f(n)/n^2          f(n)/n
      10             682.2     6.8219            68.2
     100          36,693.9     3.6694           366.9
    1000       3,099,707.8     3.0997         3,099.7
   10000     301,328,821.2     3.0133        30,132.9
  100000  30,016,609,690.5     3.0017       300,166.1
 1000000 3,000,199,315,735.7     3.0002     3,000,199.3
f(n)/n^2 -> 3 (bounded)  => Theta(n^2);  f(n)/n -> infinity => f(n) is not O(n)
Disproof of O(n): for any c, n > c/3 gives f(n) >= 3n^2 > c n.
   c=      10: n=       4 -> f(n)=178 > c*n=40   : True
   c=    1000: n=     334 -> f(n)=362,720 > c*n=334,000  : True
   c= 1000000: n=  333334 -> f(n)=333,395,822,203 > c*n=333,334,000,000  : True
```

Output – part (f): Θ(n²) constants and non-O(n) check

```
======================================================================
PART (g)  Scalability decision  (10^8 ops/s, window = 10 min = 6x10^10 ops)
======================================================================
n = 1,000,000   window budget = 60,000,000,000 operations

Module                       Complexity                 Operations        Time  Verdict
----------------------------------------------------------------------------------------
M1 Duplicate detector        Theta(n^2)            499,999,500,000    83.3 min  FAIL
M2 Rate limiter              Theta(n)                    1,048,575    10.49 ms  PASS
S2 Binary search (1 query)   Theta(log n)                      20     0.20 us  PASS
M4 Recommender - current     Theta(n^2)          1,999,999,000,000     5.56 h  FAIL
M4 Recommender - redesign    Theta(n log n)             20,931,569   209.32 ms  PASS
M5 Sales analytics           Theta(n log^2 n)          209,599,497     2.10 s  PASS
Report generator f(n)        Theta(n^2)          3,000,199,315,736     8.33 h  FAIL

Growth when the catalogue grows 100,000 -> 1,000,000 (10x):
Module                                ops @1e5          ops @1e6    ratio
M1                                4,999,950,000   499,999,500,000    100.0
M2                                      131,071         1,048,575      8.0
M4 current                         19,999,900,000 1,999,999,000,000    100.0
M4 redesign                         1,760,964        20,931,569     11.9
M5                                 14,724,490       209,599,497     14.2
Report f(n)                        30,016,609,690 3,000,199,315,736    100.0

Redesign estimate for M1 (sort by (title,seller) key, then scan neighbours):
    ~ n log2 n + n = 20,931,569 ops = 209.32 ms  (hash-set variant: ~n ops, Theta(n) expected)
chart saved -> /home/claude/CO1_AT1/Question_1/Source_Code/../Supporting_Files/growth_chart.png
```

Output – part (g): scalability table and growth factors

Conclusion

At the projected catalogue size of one million products, three components do not fit the 10-minute nightly window: the duplicate detector M1 (Θ(n²), 83 min), the current recommendation engine M4 (Θ(n²), 5.6 h) and the report generator (Θ(n²), 8.3 h). The rate limiter M2 is linear – not n log n as claimed – and, together with binary search S2, the redesigned M4 and the analytics engine M5 (Θ(n log² n)), it scales comfortably. The three quadratic modules should be redesigned to Θ(n) or Θ(n log n) before the expansion; the single most effective changes are hashing the (title, seller) key in M1 and adopting the non-replicated divide-and-conquer design in M4.